After completing part 2, you should be well underway to becoming an advanced Airflow user, being able to write complex (and testable) pipelines that optionally involve custom components and/or containers. However, depending on your interests, you may want to pick specific chapters to focus on, as not all chapters may be relevant to your use case.

6 Triggering workflows

This chapter covers

- Waiting for certain conditions to be met with sensors
- Deciding how to set dependencies between tasks in different DAGs
- Executing workflows via the CLI and REST API

In chapter 3, we explored how to schedule workflows in Airflow based on a time interval. The time intervals can be given as convenience strings (e.g., `"@daily"`), timedelta objects (e.g., `timedelta(days=3)`), or cron strings (e.g., `"30 14 * * *"`). These are all notations to instruct the workflow to trigger at a certain time or interval. Airflow will compute the next time to run the workflow given the interval and start the first task(s) in the workflow at the next date and time.

In this chapter, we explore other ways to trigger workflows. This is often desired following a certain action, in contrast to the time-based intervals, which start workflows at predefined times. Trigger actions are often the result of external events; think of a file being uploaded to a shared drive, a developer pushing their code to a repository, or the existence of a partition in a Hive table, any of which could be a reason to start running your workflow.

6.1 Polling conditions with sensors

One common use case to start a workflow is the arrival of new data; imagine a third party delivering a daily dump of its data on a shared storage system between your company and the third party. Assume we're developing a popular mobile couponing app and are in contact with all supermarket brands to deliver a daily export of their promotions that are going to be displayed in our couponing app. Currently, the promotions are mostly a manual process: most supermarkets employ pricing analysts to take many factors into account and deliver accurate promotions. Some promotions are well thought out weeks in advance, and some are spontaneous one-day flash sales. The pricing analysts carefully study competitors, and sometimes promotions are made late at night. Hence, the daily promotions data often arrives at random times. We saw data arrive on the shared storage between 16:00 and 2:00 the next day, although the daily data can be delivered at any time of the day.

Let's develop the initial logic for such a workflow (figure 6.1).

Figure 6.1 Initial logic for processing supermarket promotions data

In this workflow, we copy the supermarkets' (1–4) delivered data into our own raw storage from which we can always reproduce results. The `process_supermarket_{1,2,3,4}` tasks then transform and store all raw data in a results database that can be read by the app. And finally, the `create_metrics` task computes and aggregates several metrics that give insights in the promotions for further analysis.

With data from the supermarkets arriving at varying times, the timeline of this workflow could look like figure 6.2.

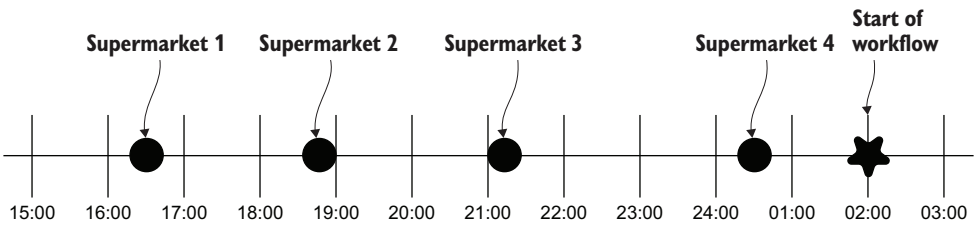

Figure 6.2 Timeline of processing supermarket promotion data

Here we see the data delivery times of the supermarkets and the start time of our workflow. Since we've experienced supermarkets delivering data as late as 2:00, a safe bet would be to start the workflow at 2:00 to be certain all supermarkets have delivered their data. However, this results in lots of waiting time. Supermarket 1 delivered its data at 16:30, while the workflow starts processing at 2:00 but does nothing for 9.5 hours (figure 6.3).

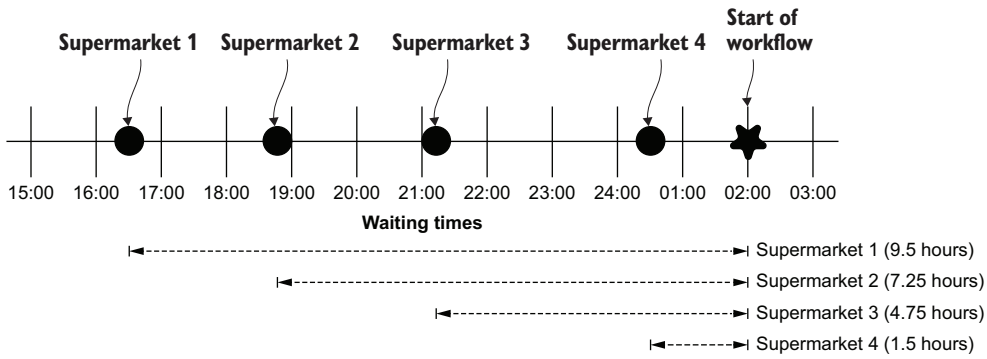

Figure 6.3 Timeline of supermarket promotion workflow with waiting times

One way to solve this in Airflow is with the help of *sensors*, which are a special type (subclass) of operators. Sensors continuously poll for certain conditions to be true and succeed if so. If false, the sensor will wait and try again until either the condition is true or a timeout is eventually reached.

Listing 6.1 A FileSensor waits for a filepath to exist

```
from airflow.sensors.filesystem import FileSensor

wait_for_supermarket_1 = FileSensor(
   task_id="wait_for_supermarket_1",
   filepath="/data/supermarket1/data.csv",
)
```

This `FileSensor` will check for the existence of /data/supermarket1/data.csv and return true if the file exists. If not, it returns false and the sensor will wait for a given period (default 60 seconds) and try again. Both operators (sensors are also operators) and DAGs have configurable timeouts, and the sensor will continue checking the condition until a timeout is reached. We can inspect the output of sensors in the task logs:

```
{file_sensor.py:60} INFO - Poking for file /data/supermarket1/data.csv
{file_sensor.py:60} INFO - Poking for file /data/supermarket1/data.csv
{file_sensor.py:60} INFO - Poking for file /data/supermarket1/data.csv
{file_sensor.py:60} INFO - Poking for file /data/supermarket1/data.csv
{file_sensor.py:60} INFO - Poking for file /data/supermarket1/data.csv
```

Here we see that approximately once a minute,[1] the sensor *pokes* for the availability of the given file. *Poking* is Airflow's name for running the sensor and checking the sensor condition.

When incorporating sensors into this workflow, one change should be made. Now that we know we won't wait until 2:00 and assume all data is available, but instead start continuously, checking if the data is available, the DAG start time should be placed at the start of the data arrival boundaries (figure 6.4).

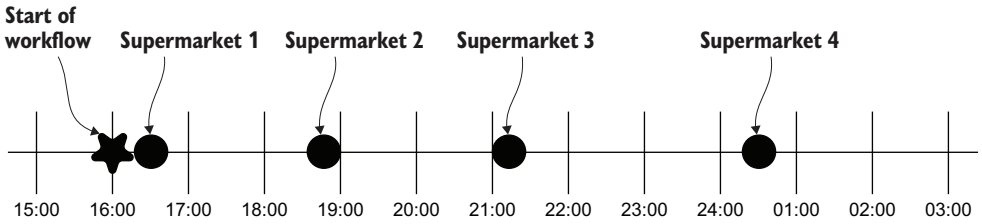

Figure 6.4 Supermarket promotions timeline with sensors

The corresponding DAG will have a task (`FileSensor`) added to the start of processing each supermarket's data and would look like figure 6.5.

Figure 6.5 Supermarket promotion DAG with sensors in Airflow

In figure 6.5, sensors were added at the start of the DAG and the DAG's `schedule_interval` was set to start *before* the expected delivery of data. This way, sensors at the start of the DAG would continuously poll for the availability of data and continue to the next task once the condition has been met (i.e., once the data is available in the given path).

[1] Configurable with the `poke_interval` argument.

Here we see supermarket 1 has already delivered data, which sets the state of its corresponding sensor to success and continues processing its downstream tasks. As a result, data was processed directly after delivery, without unnecessarily waiting until the end of the expected time of delivery.

6.1.1 Polling custom conditions

Some data sets are large and consist of multiple files (e.g., data-01.csv, data-02.csv, data-03.csv, etc.). Airflow's `FileSensor` supports wildcards to match, for example, data-*.csv, which will match any file matching the pattern. So, if, for example, the first file data-01.csv is delivered while others are still being uploaded to the shared storage by the supermarket, the `FileSensor` would return true and the workflow would continue to the `copy_to_raw` task, which is undesirable.

Therefore, we agreed with the supermarkets to write a file named _SUCCESS as the last part of uploading, to indicate the full daily data set was uploaded. The data team decided they want to check for both the existence of one or more files named data-*.csv and one file named _SUCCESS. Under the hood the `FileSensor` uses *globbing* (https://en.wikipedia.org/wiki/Glob) to match patterns against file or directory names. While globbing (similar to regex but more limited in functionality) would be able to match multiple patterns with a complex pattern, a more readable approach is to implement the two checks with the `PythonSensor`.

The `PythonSensor` is similar to the `PythonOperator` in the sense that you supply a Python callable (function, method, etc.) to execute. However, the `PythonSensor` callable is limited to returning a Boolean value: true to indicate the condition is met successfully, false to indicate it is not. Let's check out a `PythonSensor` callable checking these two conditions.

Listing 6.2 Implementing a custom condition with the PythonSensor

```
from pathlib import Path

from airflow.sensors.python import PythonSensor

def _wait_for_supermarket(supermarket_id):
    supermarket_path = Path("/data/" + supermarket_id)
    data_files = supermarket_path.glob("data-*.csv")
    success_file = supermarket_path / "_SUCCESS"
    return data_files and success_file.exists()


wait_for_supermarket_1 = PythonSensor(
    task_id="wait_for_supermarket_1",
    python_callable=_wait_for_supermarket,
    op_kwargs={"supermarket_id": "supermarket1"},
    dag=dag,
)
```

Initialize Path object.

Collect data-*.csv files.

Collect _SUCCESS file.

Return whether both data and success files exist.

The callable supplied to the `PythonSensor` is executed and expected to return a Boolean true or false. The callable shown in listing 6.2 now checks two conditions, namely

if both the data and the success file exist. Other than using a different color, the `PythonSensor` tasks appear the same in the UI (figure 6.6).

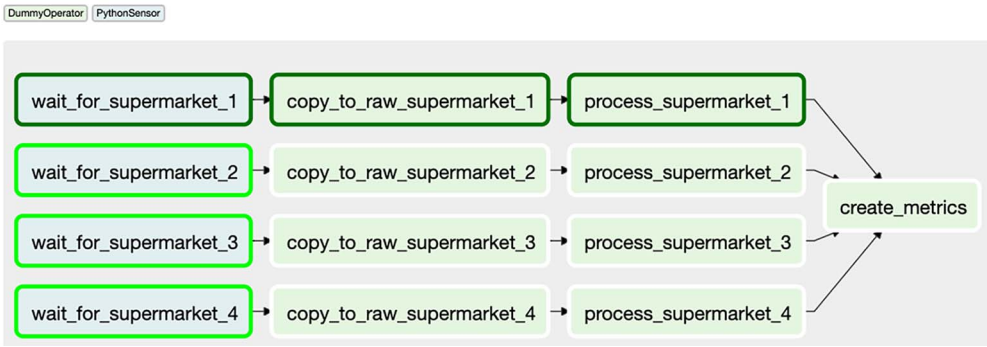

Figure 6.6 Supermarket promotion DAG using `PythonSensors` for custom conditions

6.1.2 Sensors outside the happy flow

Now that we've seen sensors running successfully, what happens if a supermarket one day doesn't deliver its data? By default, sensors will fail just like operators (figure 6.7).

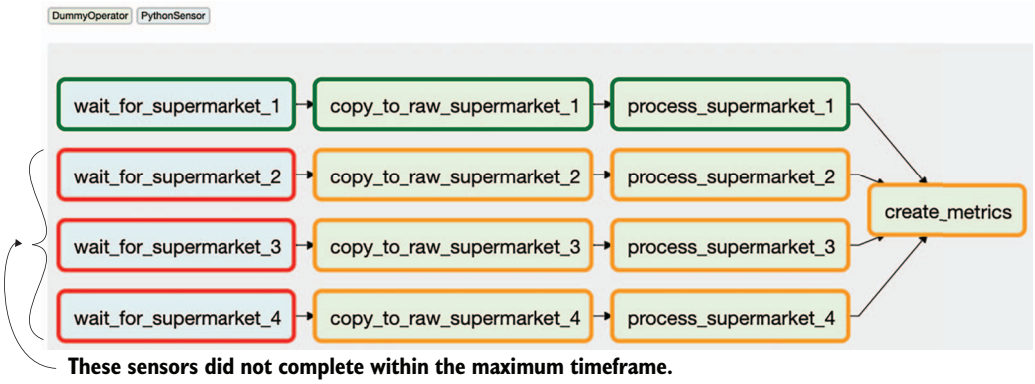

Figure 6.7 Sensors exceeding the maximum timeframe will fail.

Sensors accept a `timeout` argument, which holds the maximum number of seconds a sensor is allowed to run. If, at the start of the next poke, the number of running seconds turns out to be higher than the number set to timeout, the sensor will fail:

```
INFO - Poking callable: <function wait_for_supermarket at 0x7fb2aa1937a0>
INFO - Poking callable: <function wait_for_supermarket at 0x7fb2aa1937a0>
ERROR - Snap. Time is OUT.
```

```
Traceback (most recent call last):
  ➥ File "/usr/local/lib/python3.7/site-
     packages/airflow/models/taskinstance.py", line 926, in _run_raw_task
    result = task_copy.execute(context=context)
  ➥ File "/usr/local/lib/python3.7/site-
     packages/airflow/sensors/base_sensor_operator.py", line 116, in execute
    raise AirflowSensorTimeout('Snap. Time is OUT.')
airflow.exceptions.AirflowSensorTimeout: Snap. Time is OUT.
INFO - Marking task as FAILED.
```

By default, the sensor timeout is set to seven days. If the DAG `schedule_interval` is set to once a day, this will lead to an undesired snowball effect—which is surprisingly easy to encounter with many DAGs! The DAG runs once a day, and supermarkets 2, 3, and 4 will fail after seven days, as shown in figure 6.7. However, new DAG runs are added every day and the sensors for those respective days are started, and as a result more and more tasks start running. Here's the catch: there's a limit to the number of tasks Airflow can handle and will run (on various levels).

It is important to understand there are limits to the maximum number of running tasks on various levels in Airflow; the number of tasks per DAG, the number of tasks on a global Airflow level, the number of DAG runs per DAG, and so on. In figure 6.8, we see 16 running tasks (which are all sensors). The DAG class has a `concurrency` argument, which controls how many simultaneously running tasks are allowed within that DAG.

Listing 6.3 Setting the maximum number of concurrent tasks in a DAG

```
dag = DAG(
    dag_id="couponing_app",
    start_date=datetime(2019, 1, 1),
    schedule_interval="0 0 * * *",
    concurrency=50,
)
```

This DAG allows 50 concurrently running tasks.

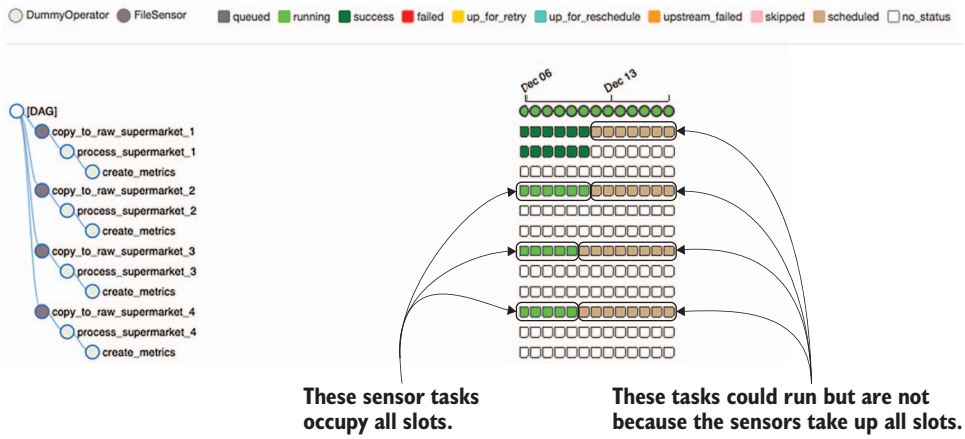

Figure 6.8 Sensor deadlock: the running tasks are all sensors waiting for a condition to be true, which never happens and thus occupy all slots.

In figure 6.8, we ran the DAG with all defaults, which is 16 concurrent tasks per DAG. The following snowball effect happened:

- Day 1: Supermarket 1 succeeded; supermarkets 2, 3, and 4 are polling, occupying 3 tasks.
- Day 2: Supermarket 1 succeeded; supermarkets 2, 3, and 4 are polling, occupying 6 tasks.
- Day 3: Supermarket 1 succeeded; supermarkets 2, 3, and 4 are polling, occupying 9 tasks.
- Day 4: Supermarket 1 succeeded; supermarkets 2, 3, and 4 are polling, occupying 12 tasks.
- Day 5: Supermarket 1 succeeded; supermarkets 2, 3, and 4 are polling, occupying 15 tasks.
- Day 6: Supermarket 1 succeeded; supermarkets 2, 3, and 4 are polling, occupying 16 tasks; two new tasks cannot run, and any other task trying to run is blocked.

This behavior is often referred to as *sensor deadlock*. In this example, the maximum number of running tasks in the supermarket couponing DAG is reached, and thus the impact is limited to that DAG, while other DAGs can still run. However, once the global Airflow limit of maximum tasks is reached, your entire system is stalled, which is obviously undesirable. This issue can be solved in various ways.

The sensor class takes an argument `mode`, which can be set to either `poke` or `reschedule` (available since Airflow 1.10.2). By default, it's set to `poke`, leading to the blocking behavior. This means the sensor task occupies a task slot as long as it's running. Once in a while, it pokes the condition and then does nothing, but still occupies a task slot. The sensor `reschedule` mode releases the slot after it has finished poking, so it *only* occupies a slot while it's doing actual work (figure 6.9).

The number of concurrent tasks can also be controlled by several configuration options on the global Airflow level, which are covered in section 12.6. In the next section, let's look at how to split up a single DAG into multiple smaller DAGs, which trigger each other to separate concerns.

6.2 Triggering other DAGs

At some point in time, more supermarkets are added to our couponing service. More and more people would like to gain insights in the supermarket's promotions and the `create_metrics` step at the end is executed only once a day, after all supermarkets' data was delivered and processed. In the current setup, it depends on the successful state of the `process_supermarket_{1,2,3,4}` tasks (figure 6.10).

We received a question from the analyst team about if the metrics could also be made available directly after processing instead of having to wait for other supermarkets to deliver their data and run it through the pipeline. We have several options

```
wait_for_supermarket1 = PythonSensor(
task_id="wait_for_supermarket_1",
python_callable=_wait_for_supermarket,
op_kwargs={"supermarket_id": "supermarket1"},
mode="reschedule",
dag=dag,
)
```

`mode="reschedule"` applies a new state "up_for_reschedule"

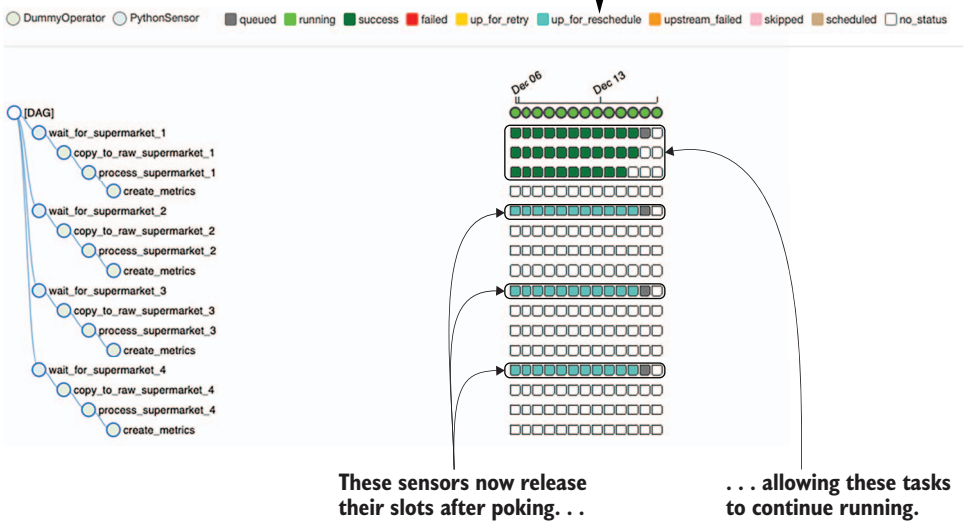

Figure 6.9 Sensors with `mode="reschedule"` release their slot after poking, allowing other tasks to run.

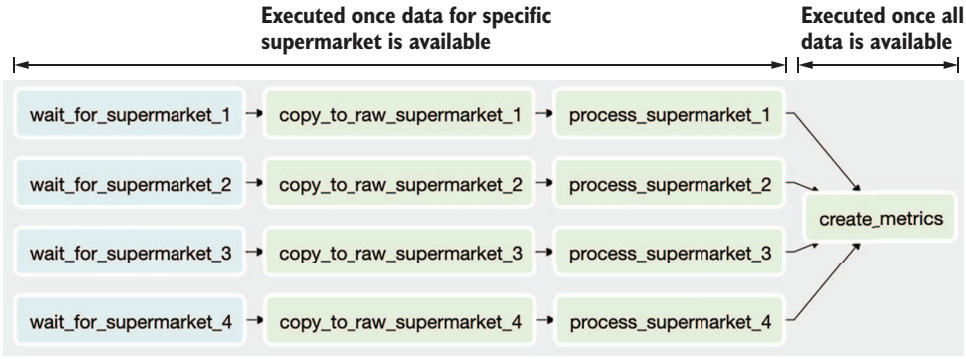

Figure 6.10 Different execution logic between the supermarket-specific tasks and the `create_metrics` task indicates a potential split in separate DAGs.

here (depending on the logic it performs). We could set the `create_metrics` task as a downstream task after every `process_supermarket_*` task (figure 6.11).

Figure 6.11 Replicating tasks to avoid waiting for completion of all processes

Suppose the `create_metrics` task evolved into multiple tasks, making the DAG structure more complex and resulting in more repeated tasks (figure 6.12).

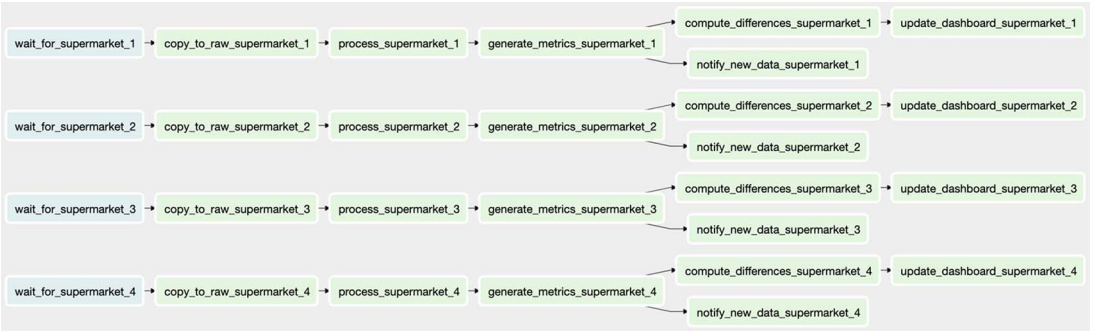

Figure 6.12 More logic once again indicates a potential split in separate DAGs.

One option to circumvent repeated tasks with (almost) equal functionality is to split your DAG into multiple smaller DAGs where each takes care of part of the total workflow. One benefit is you can call DAG 2 multiple times from DAG 1, instead of one single DAG holding multiple (duplicated) tasks from DAG 2. Whether this is possible or desirable depends on many things, such as the complexity of the workflow. If, for example, you'd like to be able to create the metrics without having to wait for the workflow to complete according to its schedule, but instead trigger it manually whenever you'd like, then it could make sense to split it into two separate DAGs.

This scenario can be achieved with the `TriggerDagRunOperator`. This operator allows triggering other DAGs, which you can apply to decouple parts of a workflow.

Listing 6.4 Triggering other DAGs using the TriggerDagRunOperator

```
import airflow.utils.dates
from airflow import DAG
from airflow.operators.dummy import DummyOperator
from airflow.operators.trigger_dagrun import TriggerDagRunOperator

dag1 = DAG(
    dag_id="ingest_supermarket_data",
    start_date=airflow.utils.dates.days_ago(3),
    schedule_interval="0 16 * * *",
)

for supermarket_id in range(1, 5):
    # ...
    trigger_create_metrics_dag = TriggerDagRunOperator(
        task_id=f"trigger_create_metrics_dag_supermarket_{supermarket_id}",
        trigger_dag_id="create_metrics",
        dag=dag1,
    )

dag2 = DAG(
    dag_id="create_metrics",
    start_date=airflow.utils.dates.days_ago(3),
    schedule_interval=None,
)
# ...
```

dag_id should align.

No schedule_interval required if only triggered

The string provided to the `trigger_dag_id` argument of the `TriggerDagRunOperator` must match the `dag_id` of the DAG to trigger. The end result is that we now have two DAGs, one for ingesting data from the supermarkets and one for computing metrics on the data (figure 6.13).

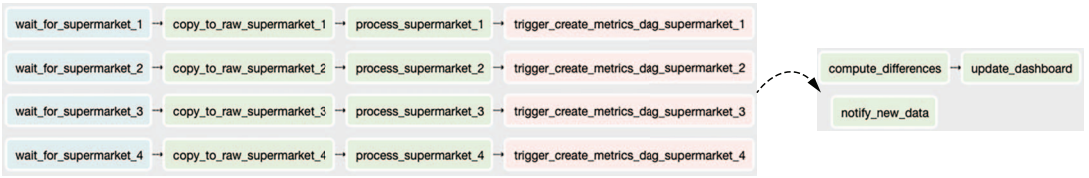

Figure 6.13 DAGs split in two, with DAG 1 triggering DAG 2 using the `TriggerDagRunOperator`. The logic in DAG 2 is now defined just once, simplifying the situation shown in figure 6.12.

Visually, in the Airflow UI there is almost no difference between a scheduled DAG, manually triggered DAG, or an automatically triggered DAG. Two small details in the

tree view tell you whether a DAG was triggered or started by a schedule. First, scheduled DAG runs and their task instances show a black border (figure 6.14).

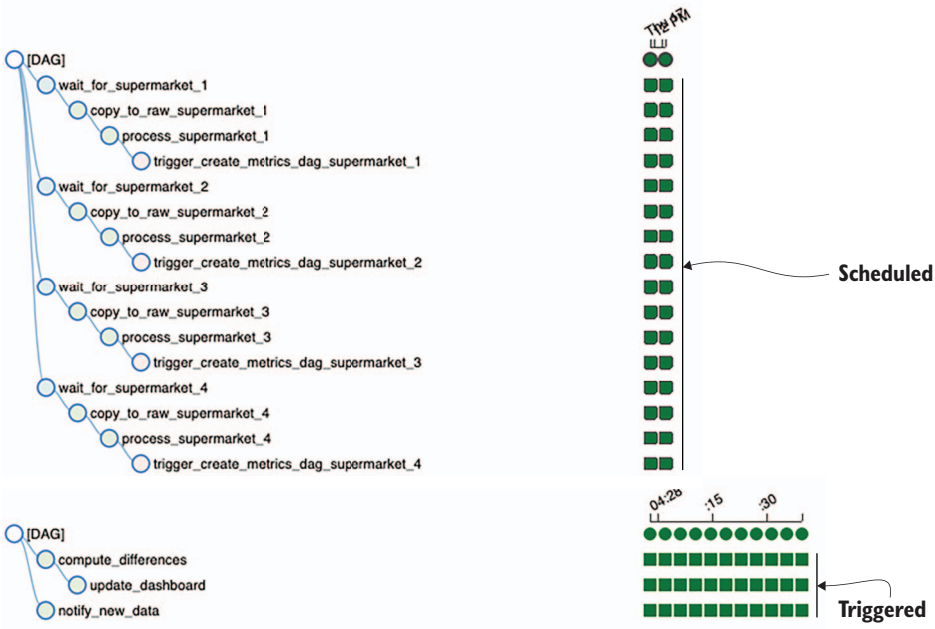

Figure 6.14 Black borders indicate a scheduled run; no borders are triggered.

Second, each DAG run holds a field `run_id`. The value of the `run_id` starts with one of the following:

- `scheduled__` to indicate the DAG run started because of its schedule
- `backfill__` to indicate the DAG run started by a backfill job
- `manual__` to indicate the DAG run started by a manual action (e.g., pressing the Trigger Dag button, or triggered by a `TriggerDagRunOperator`)

Hovering over the DAG run circle displays a tooltip showing the `run_id` value, telling us how the DAG started running (figure 6.15).

6.2.1 Backfilling with the TriggerDagRunOperator

What if you changed some logic in the `process_*` tasks and wanted to rerun the DAGs from there on? In a single DAG you could clear the state of the `process_*` and corresponding downstream tasks. However, clearing tasks *only* clears tasks within the same DAG. Tasks downstream of a `TriggerDagRunOperator` in another DAG are not cleared, so be well aware of this behavior.

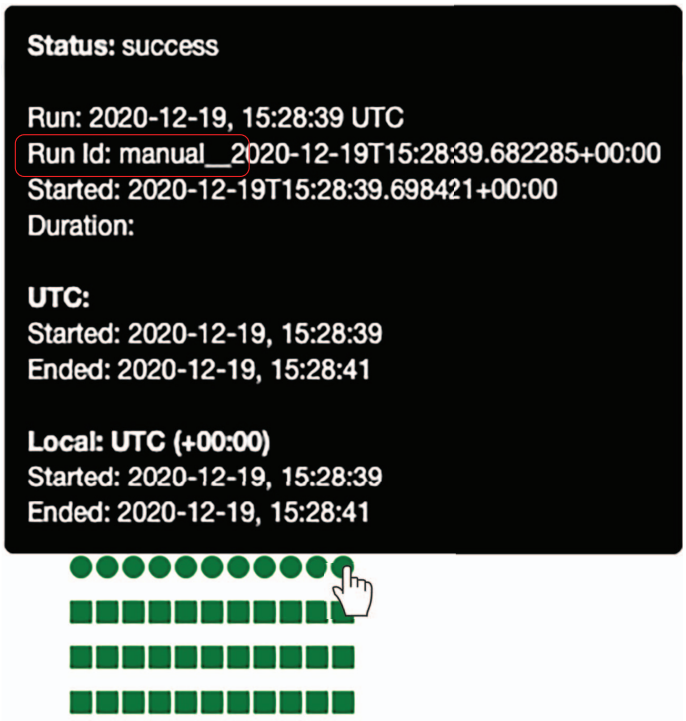

Figure 6.15 The `run_id` tells us the origin of the DAG run.

Clearing tasks in a DAG, including a `TriggerDagRunOperator`, will trigger a new DAG run instead of clearing the corresponding previously triggered DAG runs (figure 6.16).

6.2.2 Polling the state of other DAGs

The example in figure 6.13 works as long as there is no dependency from the to be triggered DAGs back to the triggering DAG. In other words, the first DAG can trigger the downstream DAG whenever, without having to check any conditions.

If the DAGs become very complex, for clarity the first DAG could be split across multiple DAGs, and a corresponding `TriggerDagRunOperator` task could be made for each corresponding DAG, as seen in figure 6.17 in the middle. Also, one DAG triggering multiple downstream DAGs is a possible scenario with the `TriggerDagRunOperator`, as seen in figure 6.17 on the right.

But what if multiple triggering DAGs must complete before another DAG can start running? For example, what if DAGs 1, 2, and 3 each extract, transform, and load a data set, and you'd like to run DAG 4 *only* once all three DAGs have completed, for example to compute a set of aggregated metrics? Airflow manages dependencies

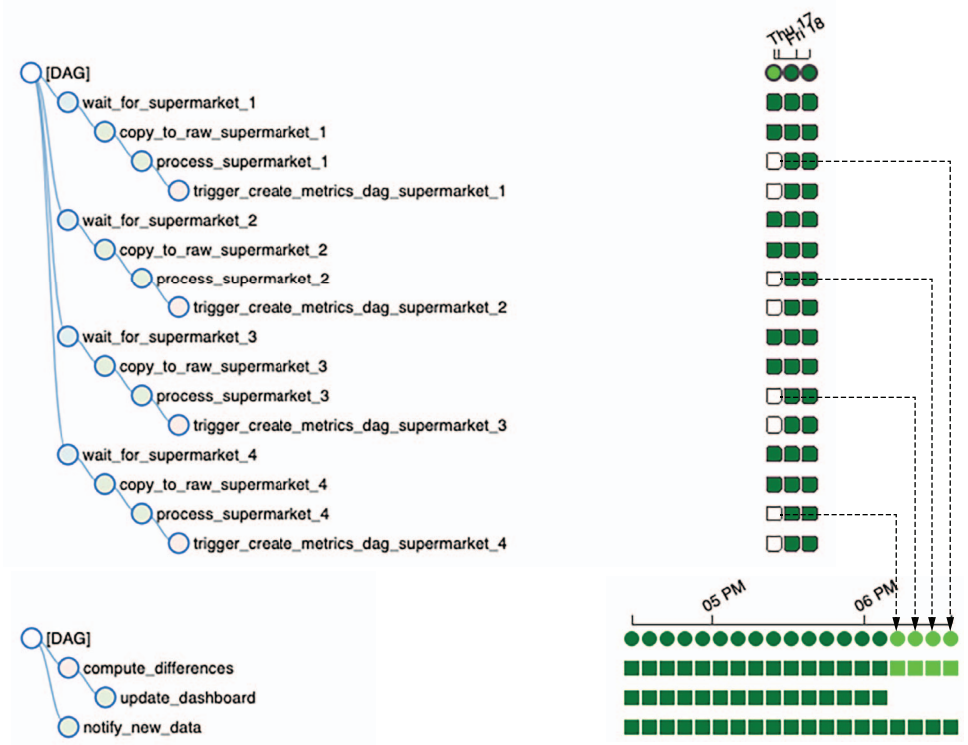

Figure 6.16 Clearing `TriggerDagRunOperators` does not clear tasks in the triggered DAG; instead, new DAG runs are created.

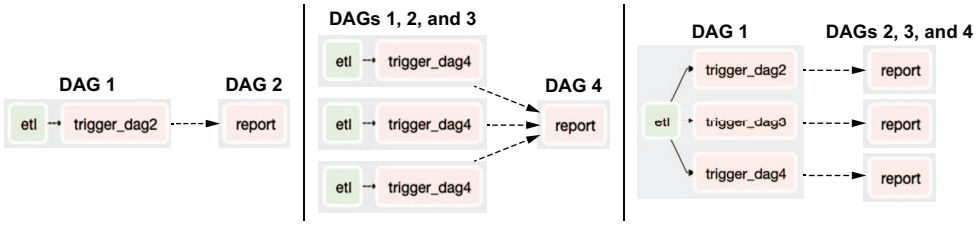

Figure 6.17 Various inter-DAG dependencies possible with the `TriggerDagRunOperator`

between tasks within one single DAG; however, it does *not* provide a mechanism for inter-DAG dependencies (figure 6.18).[2]

[2] This Airflow plug-in visualizes inter-DAG dependencies by scanning all your DAGs for usage of the `TriggerDagRunOperator` and `ExternalTaskSensor`: https://github.com/ms32035/airflow-dag-dependencies.

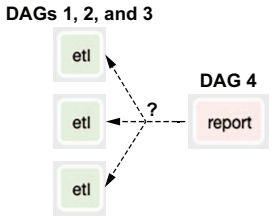

Figure 6.18 Illustration of inter-DAG dependency, which cannot be solved with the `TriggerDagRunOperator`

For this situation we could apply the `ExternalTaskSensor`, which is a sensor poking the state of tasks in other DAGs, as shown in figure 6.19. This way the `wait_for_etl_dag{1,2,3}` tasks act as a proxy to ensure the completed state of all three DAGs before finally executing the report task.

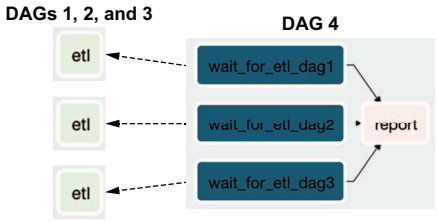

Figure 6.19 Instead of pushing execution with the `TriggerDagRunOperator`, in some situations such as ensuring completed state for DAGs 1, 2, and 3, we must pull execution toward DAG 4 with the `ExternalTaskSensor`.

The way the `ExternalTaskSensor` works is by pointing it to a task in another DAG to check its state (figure 6.20).

```
import airflow.utils.dates
from airflow  import DAG
from airflow.operators.dummy  import DummyOperator
from airflow.sensors.external_task  import ExternalTaskSensor

dag1 = DAG(dag_id="ingest_supermarket_data", schedule_interval="0 16 * * *", ...)
dag2 = DAG(schedule_interval="0 16 * * *", ...)

DummyOperator(task_id="copy_to_raw", dag=dag1) >> DummyOperator(task_id="process_supermarket", dag=dag1)

wait = ExternalTaskSensor(
    task_id="wait_for_process_supermarket",
    external_dag_id="ingest_supermarket_data",
    external_task_id="process_supermarket",
    dag=dag2,
)
report = DummyOperator(task_id="report", dag=dag2)
wait >> report
```

Figure 6.20 Example usage of the `ExternalTaskSensor`

Since there is no event from DAG 1 to DAG 2, DAG 2 is polling for the state of a task in DAG 1, but this comes with several downsides. In Airflow's world, DAGs have no notion of other DAGs. While it's technically possible to query the underlying metastore (which is what the `ExternalTaskSensor` does), or read DAG scripts from disk and infer the execution details of other workflows, they are not coupled in Airflow in any way. This requires a bit of alignment between DAGs in case the `ExternalTaskSensor` is used. The default behavior is such that the `ExternalTaskSensor` simply checks for a successful state of a task with the exact same execution date as itself. So, if an `ExternalTaskSensor` runs with an execution date of 2019-10-12T18:00:00, it would query the Airflow metastore for the given task, also with an execution date of 2019-10-12T18:00:00. Now let's say both DAGs have a different schedule interval; then these would not align and thus the `ExternalTaskSensor` would never find the corresponding task! (See figure 6.21.)

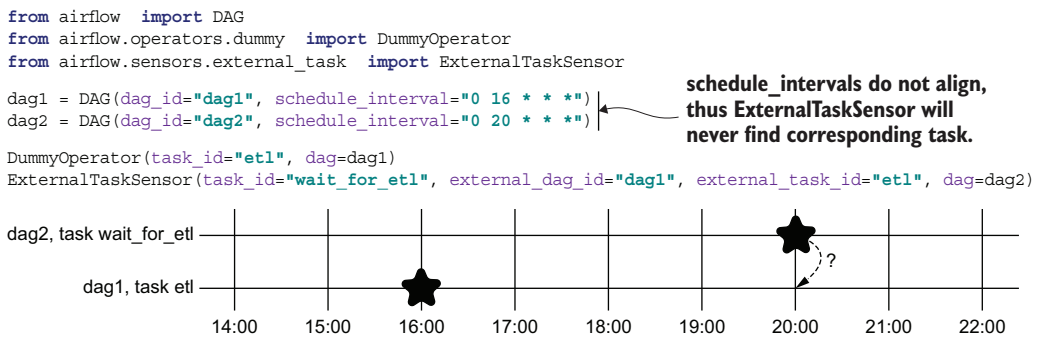

Figure 6.21 An `ExternalTaskSensor` checks for the completion of a task in another DAG, following its own `schedule_interval`, which will never be found if the intervals do not align.

In case the schedule intervals do not align we can offset, by which the `ExternalTaskSensor` must search for the task in the other DAG. This offset is controlled by the `execution_delta` argument on the `ExternalTaskSensor`. It expects a `timedelta` object, and it's important to know it operates counterintuitive from what you expect. The given `timedelta` is subtracted from the `execution_date`, meaning that a positive `timedelta` actually looks back in time (figure 6.22).

Note that checking a task using the `ExternalTaskSensor` where the other DAG has a different interval period, for example, DAG 1 runs once a day and DAG 2 runs every five hours, complicates setting a good value for `execution_delta`. For this use case, it's possible to provide a function returning a list of timedeltas via the `execution_date_fn` argument. Refer to the Airflow documentation for the details.

```
from airflow  import DAG
from airflow.operators.dummy  import DummyOperator
from airflow.sensors.external_task  import ExternalTaskSensor
dag1 = DAG(dag_id="dag1", schedule_interval="0 16 * * *")
dag2 = DAG(dag_id="dag2", schedule_interval="0 20 * * *")
DummyOperator(task_id="etl", dag=dag1)
ExternalTaskSensor(
    task_id="wait_for_etl",
    external_dag_id="dag1",
    external_task_id="etl",
    execution_delta=datetime.timedelta(hours=4),
    dag=dag2,
)
```

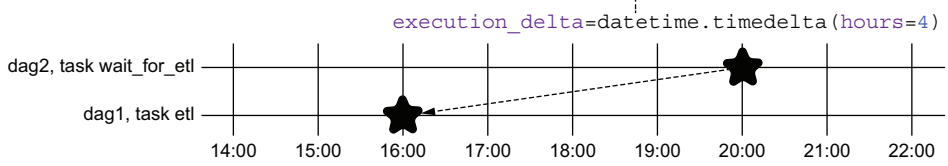

Figure 6.22 An `ExternalTaskSensor` can be offset with `execution_delta` to match with the intervals of other DAGs.

6.3 Starting workflows with REST/CLI

In addition to triggering DAGs from other DAGs, they can also be triggered via the REST API and CLI. This can be useful if you want to start workflows from outside Airflow (e.g., as part of a CI/CD pipeline). Or, data arriving at random times in an AWS S3 bucket can be processed by setting a Lambda function to call the REST API, triggering a DAG, instead of having to run sensors polling all the time.

Using the Airflow CLI, we can trigger a DAG as follows.

Listing 6.5 Triggering a DAG using the Airflow CLI

```
airflow dags trigger dag1

➥ [2019-10-06 14:48:54,297] {cli.py:238} INFO - Created <DagRun dag1 @ 2019-
    10-06 14:48:54+00:00: manual__2019-10-06T14:48:54+00:00, externally
    triggered: True>
```

This triggers `dag1` with the execution date set to the current date and time. The DAG `run id` is prefixed with "manual__" to indicate it was triggered manually, or from outside Airflow. The CLI accepts additional configuration to the triggered DAG.

Listing 6.6 Triggering a DAG with additional configuration

```
airflow dags trigger -c '{"supermarket_id": 1}' dag1
airflow dags trigger --conf '{"supermarket_id": 1}' dag1
```

This piece of configuration is then available in all tasks of the triggered DAG run via the task context variables.

Listing 6.7 Applying configuration DAG run

```
import airflow.utils.dates
from airflow import DAG
from airflow.operators.python import PythonOperator

dag = DAG(
    dag_id="print_dag_run_conf",
    start_date=airflow.utils.dates.days_ago(3),
    schedule_interval=None,
)


def print_conf(**context):
   print(context["dag_run"].conf)


process = PythonOperator(
    task_id="process",
    python_callable=print_conf,
    dag=dag,
)
```

Configuration supplied when triggering DAGs is accessible in the task context

These tasks print the conf provided to the DAG run, which can be applied as a variable throughout the task:

```
{cli.py:516} INFO - Running <TaskInstance: print_dag_run_conf.process 2019-
     10-15T20:01:57+00:00 [running]> on host ebd4ad13bf98
{logging_mixin.py:95} INFO - {'supermarket': 1}
{python_operator.py:114} INFO - Done. Returned value was: None
{logging_mixin.py:95} INFO - [2019-10-15 20:03:09,149]
     {local_task_job.py:105} INFO - Task exited with return code 0
```

As a result, if you have a DAG in which you run copies of tasks simply to support different variables, this becomes a whole lot more concise with the DAG run conf, since it allows you to insert variables into the pipeline (figure 6.23). However, note that the DAG in listing 6.8 has no schedule interval (i.e., it only runs when triggered). If the logic in your DAG relies on a DAG run conf, then it won't be possible to run on a schedule since that doesn't provide any DAG run conf.

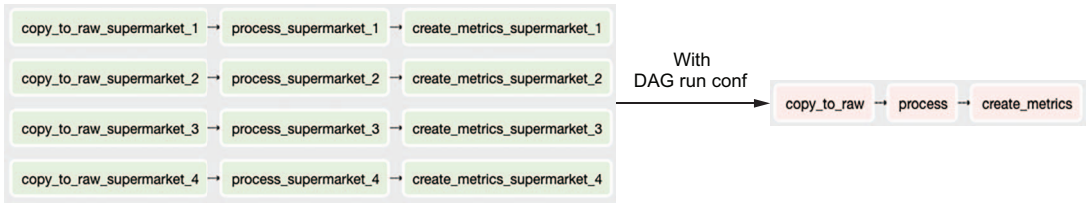

Figure 6.23 Simplifying DAGs by providing payload at runtime

Similarly, it is also possible to use the REST API for the same result (e.g., in case you have no access to the CLI but your Airflow instance can be reached over HTTP).

Listing 6.8 Triggering a DAG using the Airflow REST API

```
# URL is /api/v1

curl \
-u admin:admin \
-X POST \
"http://localhost:8080/api/v1/dags/print_dag_run_conf/dagRuns" \
-H  "Content-Type: application/json" \
-d '{"conf": {}}'

{
  "conf": {},
  "dag_id": "print_dag_run_conf",
  "dag_run_id": "manual__2020-12-19T18:31:39.097040+00:00",
  "end_date": null,
  "execution_date": "2020-12-19T18:31:39.097040+00:00",
  "external_trigger": true,
  "start_date": "2020-12-19T18:31:39.102408+00:00",
  "state": "running"
}


curl \
-u admin:admin \
-X POST \
"http://localhost:8080/api/v1/dags/print_dag_run_conf/dagRuns" \
-H  "Content-Type: application/json" \
-d '{"conf": {"supermarket": 1}}'

{
  "conf": {
    "supermarket": 1
  },
  "dag_id": "listing_6_08",
  "dag_run_id": "manual__2020-12-19T18:33:58.142200+00:00",
  "end_date": null,
  "execution_date": "2020-12-19T18:33:58.142200+00:00",
  "external_trigger": true,
  "start_date": "2020-12-19T18:33:58.153941+00:00",
  "state": "running"
}
```

Sending a plaintext username/password is not desirable; consult the Airflow API authentication documentation for other authentication methods.

The endpoint requires a piece of data, even if no additional configuration is given.

This could be convenient when triggering DAG from outside Airflow, for example from your CI/CD system.